Software Version Control with Git and GitHub

A practical guide to repositories, workflows, and tools

Dott. Angelo Porrello

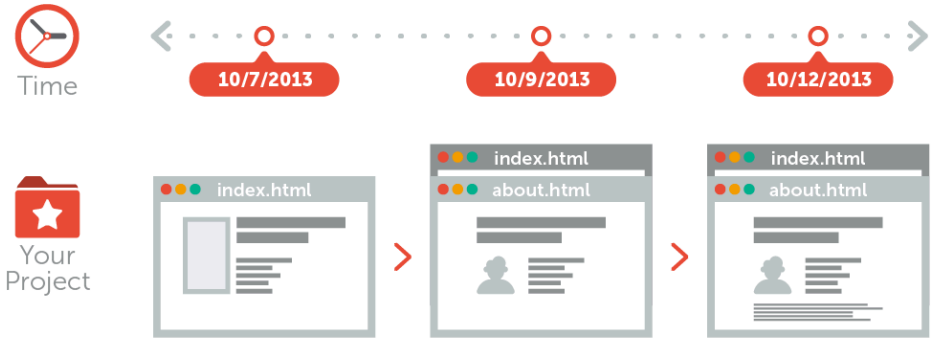
Ingegneria del Software e Lab. – Corso di Laurea Triennale Ingegneria Informatica

Agenda

Version Control Systems

Version Control System

A Version Control System (VCS) is a tool that records changes to a file or set of files over time so that it is possible to recall specific versions later.



Version Control System

A **Version Control System (VCS)** is a tool that records and manages changes to files over time, allowing developers to:

- **Track** modifications and maintain a complete history of a project;
- **Revert** to previous versions when necessary;
- **Collaborate** efficiently without overwriting each other's work;
- **Experiment safely** using branches and merging;
- **Identify authorship** of changes and review contributions.

In modern software development, VCS tools like **Git** are essential to ensure code quality, team coordination, and long-term project maintainability.

Local Version Control Systems

A first trivial way to keep track of project changes is to manually create **time-stamped copies** of the entire project directory during development.

```
$ ls -l
drwxr-xr-x project_2023-11-01/
drwxr-xr-x project_2023-11-05/
drwxr-xr-x project_2023-11-12/
drwxr-xr-x project_2023-11-20_final/
drwxr-xr-x project_2023-11-20_final2/
```

This approach is very common because it is simple, but it is also highly **error prone**.

Why?

Local Version Control Systems

A first trivial way to keep track of project changes is to manually create **time-stamped copies** of the entire project directory during development.

```
$ ls -l
drwxr-xr-x project_2023-11-01/
drwxr-xr-x project_2023-11-05/
drwxr-xr-x project_2023-11-12/
drwxr-xr-x project_2023-11-20_final/
drwxr-xr-x project_2023-11-20_final2/
```

This approach is very common because it is simple, but it is also highly **error prone**.

Why?

It is easy to forget which directory contains the changes of interest, to overwrite files by mistake, or to lose track of the correct version to edit.

Local Version Control Systems

A direct extension of the manual copy approach was the use of a **simple local database** to record and retrieve all changes made to files under revision control.

These systems offered several improvements: automatic storage of file versions in a structured format; basic operations such as retrieving older revisions or comparing changes; etc.

Local Version Control Systems

However, these early tools were designed for a *single developer* working on a *single machine*.

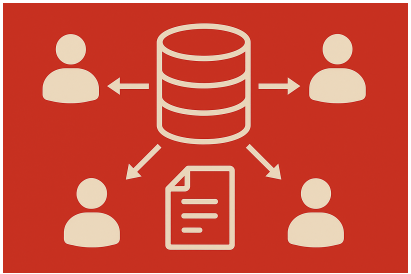
No support for collaboration. Multiple people could not safely work on the same project at the same time, leading to conflicts, overwrites, and the inability to merge contributions.

Centralized Version Control Systems

A **Centralized Version Control System (CVCS)** uses a **single central server** that stores all versioned files.

Developers act as *clients* and *check out* files from this shared repository.

Easy to maintain: one central location for all project data.



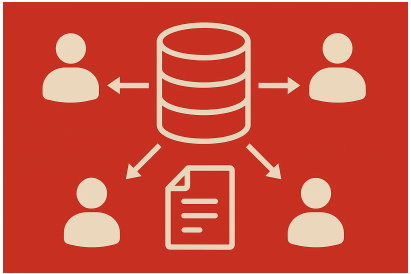
Centralized Version Control Systems

A **Centralized Version Control System (CVCS)** uses a **single central server** that stores all versioned files.

This model is often referred to as **Subversion (SVN)**.

Developers interact with the central server using two main operations:

- `update`: download the latest version from the server.
- `commit`: upload your changes to the central repository.



Centralized VCS: Limitations

Despite their simplicity, Centralized Version Control Systems suffer from **important limitations**:

Centralized VCS: Limitations

Despite their simplicity, Centralized Version Control Systems suffer from **important limitations**:

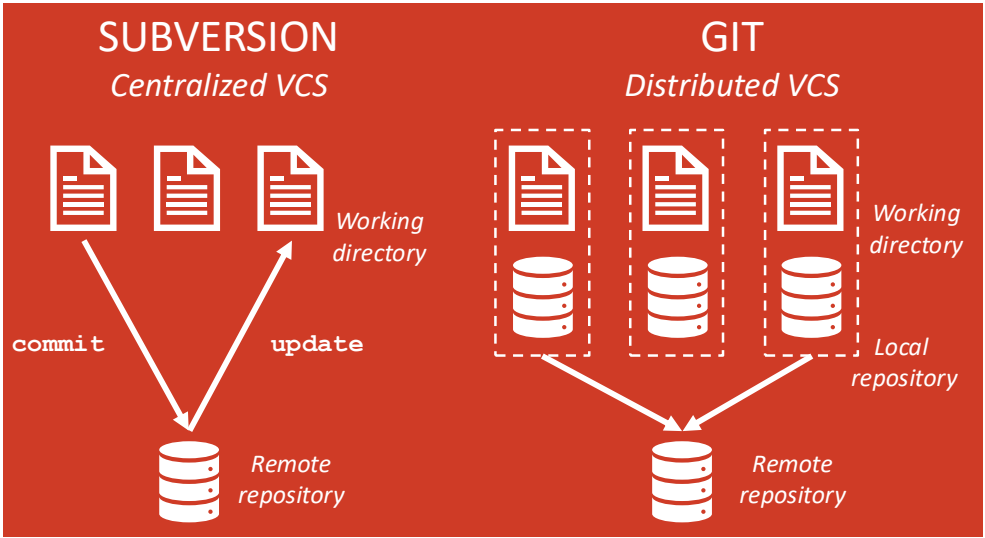
- **Single point of failure**: if the central server crashes, the entire team is blocked.
- **Limited offline work**: most operations require constant server connectivity.
- **Performance bottleneck**: all developers rely on the same shared server.

Distributed Version Control Systems

Distributed Version Control Systems (DVCS) take a different approach: developers do not merely check out the latest snapshot of a project.

- **Full local mirror**: each client obtains a complete copy of the repository (*local repository*), including its entire history.
- **No central point of failure**: if a server goes down, any client repository can be used to restore it.
- **Every clone is a backup**: each developer holds a full replica of all project data.

Centralized VCS vs. Distributed VCS



git

What is Git?

Git is an open-source project originally developed in 2005 by **Linus Torvalds** to support Linux kernel development.

Originally, Linus Torvalds used **no revision control system**. Contributors sent patches via Usenet or the mailing list, and Linus applied everything by hand.

Existing tools such as **CVS** and later **Subversion** were available, but they had serious limitations (per-file tracking, slow merges, race conditions).

Linus disliked both systems and refused to use them for kernel development.

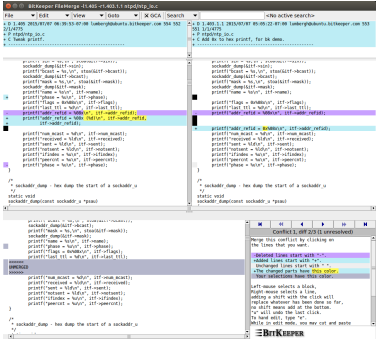
BitKeeper: A Controversial Choice

In 2002, Linus adopted **BitKeeper**, a proprietary distributed VCS. This shocked the open-source community.

BitKeeper offered:

- fast and easy distributed development,
- simple forks and merges,
- reduced load on Linus as sole integrator.

But it came with strict license restrictions and caused long-lasting tension between Linus and many kernel developers.



The Creation of Git

In 2005, BitKeeper revoked its free license. The Linux kernel suddenly had **no usable VCS**.

Linus paused kernel development and wrote **Git** in about **10 days**:

- designed for extreme **speed** and scalability,
- fully **distributed**,
- robust and secure.

Within days Git became self-hosting, within weeks it was ready for kernel development, and within months it reached full functionality.

It quickly became the standard for open-source development worldwide.

What is Git?

Git is a distributed version control system (DVCS) that allows each developer to have a complete copy of the repository and its full history.

Its main purpose is to **track changes**, enable **collaborative development**, and manage **branching and merging** efficiently and safely.

Git from the Command Line

Git is primarily a **command-line tool**.

git clone

```
$ git clone https://github.com/user/project.git   # download a repository
```

git commit

```
$ git commit -m "Add feature X"                    # save a new version locally
```

git pull

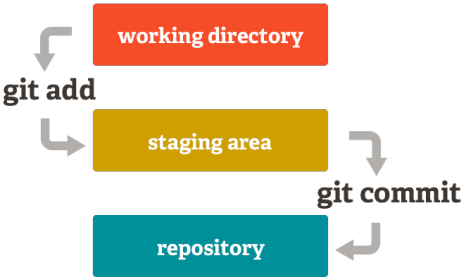
```
$ git pull origin main                             # fetch + merge updates
```

The Purpose of Git

The purpose of Git is to manage a project, or a set of files, as they change over time. Git stores this information in a data structure called a **repository**.

Git workflow consists of three steps:

1. Introduce changes on a file in the **working directory**.
2. Add the change to the **staging area** (e.g., using `git add`).
3. Make the change "permanent" by saving it in the **repository** (e.g., using `git commit`).

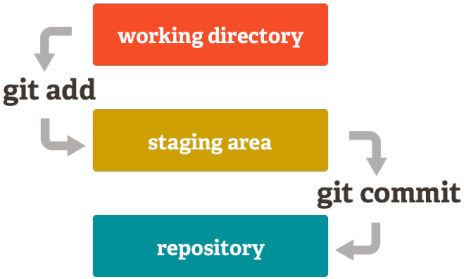


The Purpose of Git

Working directory. The place where you edit files. It contains the project's current state as seen in your filesystem.

Staging area (index). An intermediate area where you collect the changes you want to include in the next commit. It lets you prepare and organize what will be recorded.

Repository. The internal database where Git permanently stores the full history of commits, including snapshots, metadata, and version tracking information.



Git Snapshots

Snapshot. The mechanism Git uses to keep track of your project's history.

- Records what all your files look like at a specific point in time.
- You choose **when** to take a snapshot and **which files** to include.
- You can revisit any snapshot later.
- Future snapshots remain available as independent points in history.

Git Commit

Commit. A commit is the **act of saving a snapshot** into the repository's history. It includes the snapshot plus metadata such as author, timestamp, and message.

- Every commit becomes a permanent part of the repository's timeline.
- It captures the state of the tracked files at a specific moment.
- Each commit includes a **message** describing how the files changed since the previous snapshot.
- A Git project is essentially a sequence of commits that form its history.

A **snapshot** is like taking a **photo** of your project, while a **commit** is like placing that photo into an **album**, annotated with date and description.

Example: `git log`

AImage Lab
University of Modena and Reggio Emilia

```
commit a12f4c9de8ab719c33fa8c1df55a9012bbcd102a (HEAD -> main, origin/main)
Author: Alex Rover <alex.rover@example.com>
Date:   Mon Feb 10 09:42:13 2025 +0100

    Add initial project structure

commit 7bd9c182ff03aa4e92e4c8dc0ab5579efcc912e3
Merge: a12f4c9 52aa91d
Author: Sam Nightfall <sam.nightfall@example.org>
Date:   Mon Feb 10 09:12:47 2025 +0100

    Merge feature branch

commit 52aa91d3c927deabf334df22ff8c0ab4d229ab17
Author: Mira Stone <mira.stone@fantasy.net>
Date:   Fri Feb 07 17:33:21 2025 +0100

    Implement data processing module
```

What is a Repository?

A **repository** is the place where Git stores **all your files** and the **entire history** of those files.

It contains:

- all your **commits**, representing snapshots over time;
- all the metadata that tracks how the project evolved.

A repository is where all your **hard work** is safely stored. It can live:

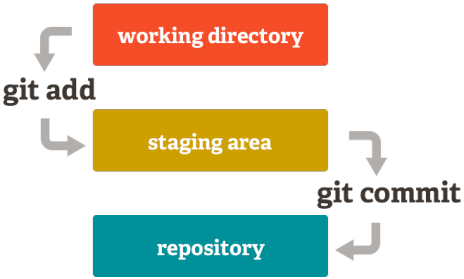
- on your **local machine**, or
- on a **remote server** (e.g., GitHub, GitLab, Bitbucket).

Key Concepts in Git

Staging area (index). A preparation zone where you choose which changes will be included in the next commit. Controlled with commands like `git add`.

Repository. Git's internal database that permanently stores commits (each a snapshot of your project's history).

Commit. The act of saving a snapshot to the repository, typically with a message describing what changed.



Remote Repositories and GitHub

A **remote repository** is a version of your project stored on a server accessible over the network. It enables collaboration, sharing, and synchronization between developers.

GitHub is a web-based platform that hosts Git repositories and enables collaboration with anyone online.

GitHub adds features on top of Git, including:

- a graphical web UI,
- documentation hosting,
- issue and bug tracking,
- feature requests,
- pull requests and code review,
- project and team management tools.

More info: `https://github.com/`

Git vs GitHub

Git: a distributed version control system that runs locally and tracks changes to your files.

GitHub: an online hosting and collaboration platform built around Git.

Other Git Hosting Platforms

Git repositories can be hosted on several online platforms, not only on GitHub.

Bitbucket

- Web-based Git hosting service by Atlassian.
- Free plans include unlimited private repositories (up to 5 users).
- Integrates with Jira, CI/CD pipelines, and team workflows.
- More info: `https://bitbucket.org/product`

GitLab

- Open-source Git hosting and full DevOps platform.
- Provides issues, merge requests, CI/CD, security scans.
- Available as cloud service or self-hosted.

Creating a GitHub Repository

What Is a GitHub Repository?

A GitHub repository is an online version of your project hosted on GitHub. It stores:

- your code and files,
- the full Git history (commits, branches, tags),
- collaboration tools (pull requests, issues, actions CI/CD),
- access control and permissions.

A repository may start:

- from scratch on GitHub,
- from an existing local Git project that you upload,
- by cloning someone else's repository.

Installing Git and Configuring Your User

Install Git

- **Linux (Debian)**

  ```
  $ sudo apt-get install git
  ```

- **macOS**
  - Download from: `https://git-scm.com/download/mac`
- **Windows**
  - Download from: `https://git-scm.com/download/win`

Configure your Git identity

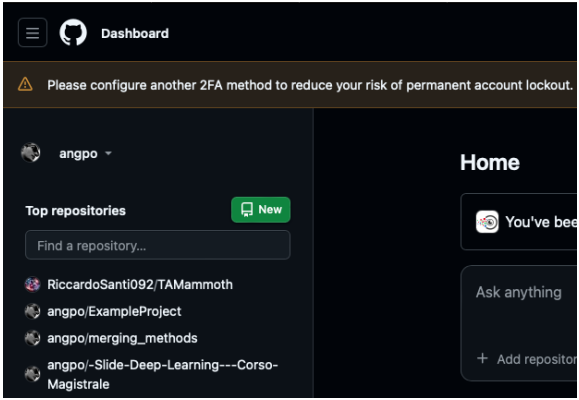
```
$ git config --global user.name "Your Name"
$ git config --global user.email "your@email.com"
```

These values appear in every commit you create.

Creating a New Repository on GitHub

To create a new repository using the GitHub web interface:

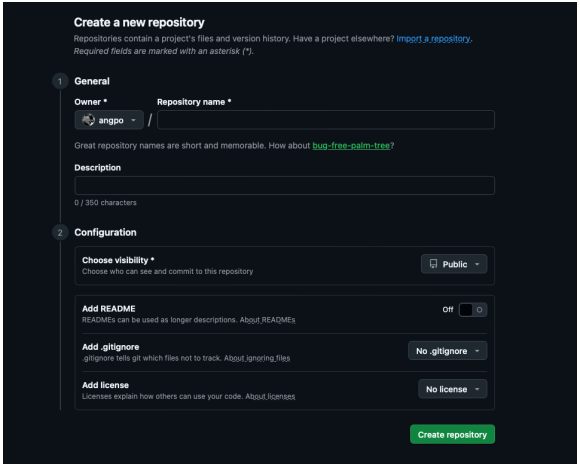
1. Sign in at `github.com`.
2. Click **New** or + → **New repository**.
3. Choose a **repository name**.
4. Select visibility: **Public** or **Private**.
5. Optionally initialize with:
   - a `README.md`,
   - a `.gitignore`,
   - a license file.
6. Click **Create repository**.



Creating a New Repository on GitHub

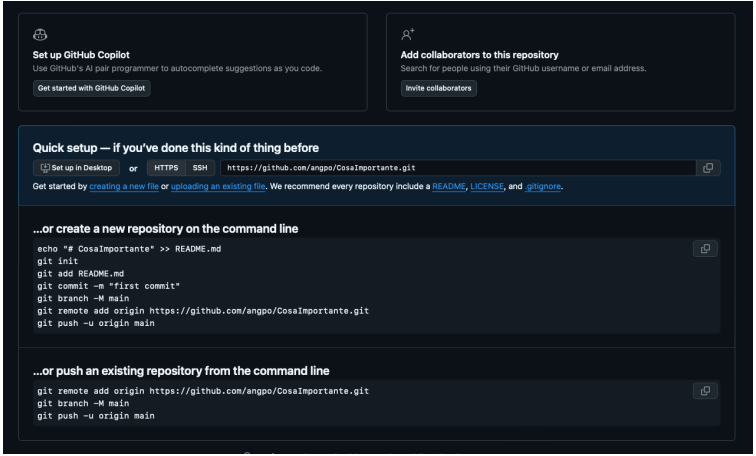
To create a new repository using the GitHub web interface:

1. Sign in at `github.com`.
2. Click **New** or **+** → **New repository**.
3. Choose a **repository name**.
4. Select visibility: **Public** or **Private**.
5. Optionally initialize with:
   - a `README.md`,
   - a `.gitignore`,
   - a license file.
6. Click **Create repository**.



After creation

After creation, GitHub shows instructions for connecting or pushing your project.

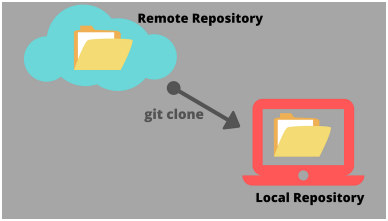


GitHub Workflow

Cloning a Repository

Cloning is the act of copying an entire repository from a remote server to your local machine.

Cloning allows teams to work together: each developer receives a complete copy of the project, including its full history.



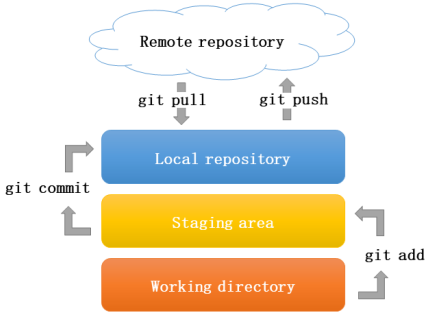
Example:

```
$ git clone https://github.com/example/project.git
```

Pulling and Pushing Changes

The process of downloading commits that do not exist on your machine from a remote repository is called **pulling changes**. This retrieves updates made by other collaborators.

The process of adding your local commits to the remote repository is called **pushing changes**. This shares your work with the rest of the team.



Pulling and Pushing Changes

Pulling. The command `git pull` downloads commits from a remote repository and integrates them into your local branch.

Pushing. The command `git push` uploads your local commits to the remote repository.

Terminology:

- **origin**: the default name of the remote repository.
- **main**: the branch you pull from or push to (historically called **master**, renamed for inclusive terminology).
- Both can be omitted: Git assumes the **current branch** and the **default remote (origin)**.

Examples:

```
$ git pull origin main
```

```
$ git push origin main
```

Implicit versions:

```
$ git pull
$ git push
```

From Local Changes to `git push`

The process of sending your work to a remote repository involves four steps:

1. Modify one or more files in the **working directory**.
2. Add the changes to the **staging area** using `git add`.
3. Create a **commit** that records a snapshot of the project.
4. Upload the commit to the remote repository using `git push`.

Example:

```
$ git add A.java
$ git commit -m "Fix bug in parser"
$ git push origin main
```

Resolving Conflicts in Git

Scenario: Local Changes vs `git pull`

Consider the following situation:

- Your local repository contains two files: `A.java` and `B.java`.
- You edit `A.java` locally but do not commit your changes.
- Then you try to run:

```
$ git pull
```

What happens?

Scenario: Local Changes vs `git pull`

Consider the following situation:

- Your local repository contains two files: `A.java` and `B.java`.
- You edit `A.java` locally but do not commit your changes.
- Then you try to run:

  ```
  $ git pull
  ```

What happens?

```
$ git pull
error: Your local changes to the following files would be overwritten by merge:
    A.java
Please commit your changes or stash them before you merge.
Aborting
```

Scenario: Local Changes vs `git pull`

Consider the following situation:

- Your local repository contains two files: `A.java` and `B.java`.
- You edit `A.java` locally but do not commit your changes.
- Then you try to run:

```
$ git pull
```

What happens?

- Git attempts to download and merge updates from the remote.
- If remote changes affect the **same lines** of `A.java`, Git cannot safely merge and stops the pull.
- Git warns you that you have **local modifications** that would be overwritten.
- You must commit or stash your changes before pulling.

What Happens When You Pull?

The result of `git pull` depends on where the changes occur.

Case 1: Changes in the same file. Git attempts to merge automatically, but in practice it will **very often fail** if the edits are close or affect the same block of code. This leads to a merge conflict that must be resolved manually.

Case 2: Changes in different files. Git can merge the histories cleanly and performs a **fast-forward merge**, meaning your branch pointer simply moves forward to include the new commits (without creating a merge commit).

Example of fast-forward:

```
$ git pull
Updating a12f4c9..b73de21
Fast-forward
 B.java | 5 ++++-
```

When Can a `git push` Fail?

A push may fail when Git cannot safely update the remote branch because the two histories do not match.

Typical reasons:

- **Local branch is behind.** Someone pushed new commits to the remote. Your branch is missing those commits and must be updated first.
- **Local and remote histories have diverged.** You have commits the remote does not have, and vice-versa. Git cannot fast-forward the update.
- **Permission or authentication issues** Missing rights, invalid token, or SSH key problems.

Examples:

```
# Local is behind
remote: A -- B -- C
local : A -- B
(push fails)

# Histories diverged
remote: A -- B -- C
local : A -- B -- D
(push fails)
```

Best Practice: Avoid Conflicts Rather Than Fix Them

Experienced developers and well-organized teams try to **avoid conflicts before they happen**.

- Perform **frequent pulls** to stay up to date.
- Make **small, focused commits** to reduce overlap.
- Keep branches **short-lived** to minimize divergence.
- Avoid modifying the **same part of the code** in parallel (coordinate within the team).

Understanding the Repository State

Knowing the state of your repository helps you avoid conflicts and handle them consciously. Before committing or merging, always check:

`git status` — shows current changes, staged files, and whether your branch is ahead/behind the remote.

`git log` — shows commit history, helping you understand what changed and when.

`git diff` — shows the exact line-by-line differences between your changes and the last commit.

Being aware of the repository state reduces merge conflicts and helps you resolve issues confidently.

git log

You can inspect the **history of commits** in the current branch with the command `git log`.

Use it to understand:

- what happened in the past,
- commit order and messages,
- who changed what and when.

```
$ git log --oneline
8b3f91c  Working version
c12e0ab  Refactor module
9f4a812  Add new feature
```

What Does `git status` Do?

The command `git status` shows the current state of your working directory and staging area.

It tells you:

- which files have been modified,
- which files are staged (ready to be committed),
- which files are untracked,
- whether your branch is ahead or behind the remote, etc.

```
$ git status
On branch main

Your branch is ahead of 'origin/main' by 1 commit.
    (use "git push" to publish your local commits)

Changes not staged for commit:
    (use "git add <file>..." to update what will be
    modified: A.java

Untracked files:
    (use "git add <file>..." to include in what will
    notes.txt
```

Using `git diff` to Inspect Changes

Before resolving a conflict, it's important to understand **what has changed** in your files.

The command `git diff` shows the differences between:

- your working directory and the last commit,
- your local branch and the remote branch,
- or any two commits you want to compare.

`git diff` highlights exactly which lines have been added, removed, or modified.

Example:

```
$ git diff A.java
diff --git a/A.java b/A.java
--- a/A.java
+++ b/A.java
@@ -12,7 +12,7 @@
-    System.out.println("Old message");
+    System.out.println("New message");
```

Ignoring Files with `.gitignore`

What `.gitignore` does

The file `.gitignore` tells Git which files or directories should not be tracked or included in commits.

Common use cases:

- temporary or generated files,
- build outputs,
- editor/OS artifacts,
- local configuration or secrets.

Ignored files are skipped by `git add` and remain untracked.

Example `.gitignore`

```
# Build artifacts
*.o
*.class

# Dependencies
node_modules/

# Editor / OS files
.DS_Store
*.swp

# Local secrets
.env
```

Discarding Local Changes After a Conflict

If a merge conflict occurs on files that you modified locally but **you do not want to keep those changes**, Git allows you to restore the files to their original state.

Modern and recommended approach:

```
$ git restore <file>
```

Historical (still supported) command:

```
$ git checkout <file>
```

Both commands discard the local modifications and restore the version from the last committed state, allowing the merge to continue cleanly.

Divergent Branches: When Push or Pull Cannot Proceed

A problem occurs when your local branch and the remote branch have followed **different histories**.

This happens when:

- someone has pushed new commits to `origin/main`, and
- you have also created new commits locally.

```
# Histories diverged
remote: A -- B -- C
local : A -- B -- D
(push fails)
```

In this situation, Git cannot simply update your branch because doing so would overwrite part of someone's work or part of your own.

This state is called a **divergence**.

Why Git Stops and Refuses to Continue

When Git detects divergent histories, it cannot automatically decide how to combine the two lines of development.

Git wants to avoid losing your commits, overwriting someone else's commits, altering history without your consent.

Therefore, Git **pauses the operation** (pull or push) and asks you to choose **how the histories should be reconciled**.

Example of Divergent Branch Warning


```
(base) aporrello@MacBook-Air-di-Angelo ExampleProject % git pull
remote: Enumerating objects: 5, done.
...
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
(base) aporrello@MacBook-Air-di-Angelo ExampleProject %
```

What Git Expects You to Choose

To continue, Git needs to know **how to reconcile** your local history with the remote history.

You must choose *one strategy* that tells Git. Git will not proceed until you specify your preference — either for this repository or globally.

The detailed strategies (merge, rebase, fast-forward) will be explained in the next slides.

Merge Strategy

Merge is the strategy where Git combines the remote history with your local history by creating a new **merge commit**.

What happens:

- Git fetches the remote commits.
- Detects that the histories have diverged.
- Attempts an automatic merge.
- You manually resolve conflicts (if any).
- Git creates a merge commit with *two parents*.

Result: Both histories remain intact and converge into a unified timeline.

Example representation:

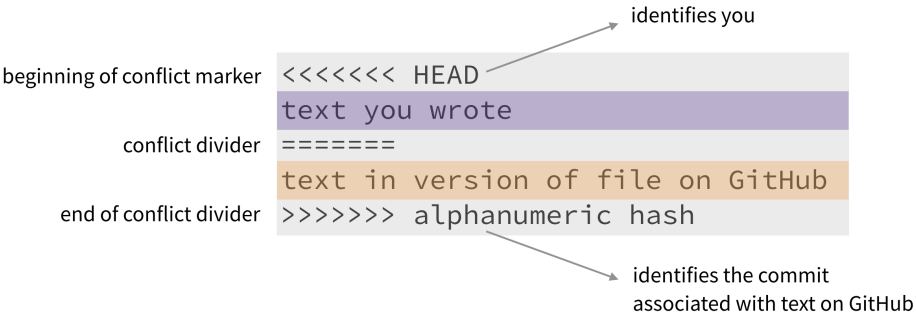
```
# Local history
A --- B --- D
        \
# Remote history
         C

# After merge
A --- B --- D
        \   \
         \   M  <-- merge commit
          \ /
           C
```

How Git Signals Merge Conflicts

When Git cannot automatically merge two versions of a file, it inserts **conflict markers** directly inside the file.

Structure of a conflict:



You must edit the file manually to keep one version, the other, or combine them, then remove the markers and complete the merge.

Rebase Strategy

Rebase is the strategy where Git rewrites your local history so that your commits appear *on top of* the remote commits.

What happens:

- Git temporarily removes your local commits.
- Updates your branch to match the remote branch.
- Reapplies your commits one by one on the updated base.

Result: A clean, **linear history** without merge commits. **Your work appears as if it was developed after the remote changes.**

Example representation:

```
# Before rebase
A --- B --- D    (local)
        \
         C --- E   (remote)

# After rebase
A --- B --- C --- E --- D'
                (rebased commit)
```

Fast-Forward Strategy

A **fast-forward** happens when your local branch has **no commits of its own** and is simply behind the remote branch.

What happens:

- Git sees that the remote branch is ahead.
- Your local branch has **no additional commits**.
- You also have **no uncommitted changes** in your working directory.
- Git can safely move the branch pointer forward with no merge commit.

Important: If you have **any local modifications** (committed or uncommitted), fast-forward **cannot happen**. Git will stop to avoid overwriting your work.

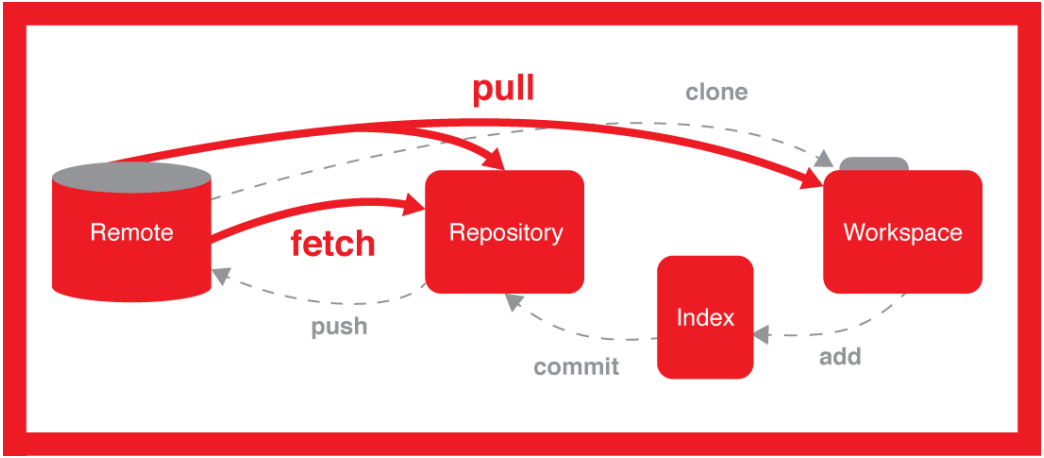
Example representation:

```
# Before fast-forward
remote: A --- B --- C
local : A --- B

# Fast-forward update
local pointer moves forward:

A --- B --- C
        ^
     now here
```

Overview


pull
clone
Remote
fetch
Repository
Workspace
push
Index
commit
add

`git pull` vs `git fetch`

`git pull` Downloads new commits from the remote repository *and immediately* tries to integrate them into your current branch (via merge or rebase).

`git fetch` Downloads new commits from the remote repository **but does not merge them**. It updates only the remote-tracking references (e.g., `origin/main`).

Key difference:

- `git fetch` = **safe update** of remote info (no changes to your files).
- `git pull` = **fetch + merge/rebase** (your working directory may change).

When to use which:

- Use `git fetch` to see what changed *before* integrating it.
- Use `git pull` when you're ready to update your local branch.

Using `git stash`

git stash temporarily saves your uncommitted changes and restores a clean working directory.

When is it useful?

- When a `git pull` is blocked because your local changes would be overwritten.
- When you want to "park" unfinished work without committing it.

Result: You safely store your in-progress work while Git performs operations that require a clean working directory.

Typical workflow:

```
# Save your local changes
$ git stash

# Update or switch branch
$ git pull
$ git switch another-branch

# Restore your changes
$ git stash pop
```

Going Back to a Previous Commit

Sometimes the software stops working as expected after recent changes. To investigate, you can temporarily return to an earlier commit and run the code from that point.

1. Identify an older commit

```
$ git log --oneline
9f4a812  Add new feature (current broken state)
c12e0ab  Refactor module
8b3f91c  Working version  <-- we want to test this
```

2. Check out the previous commit (detached HEAD mode)

```
$ git checkout 8b3f91c <-- Now your working directory reflects the project as it was at that c
```

3. Run and test the software

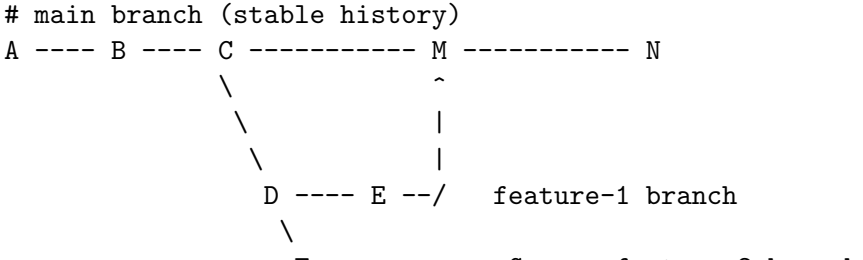
If the software works again, the bug was introduced *after* this commit. If it is still broken, the issue was already present.

Branching

What Is a Branch in Git?

A **branch** is an independent line of development. It allows you to experiment, add features, or fix bugs without affecting the `main` branch.

Typical branching workflow on GitHub:



Why Branching?

Branches let developers:

- work on features in isolation,
- open **pull requests** to merge their work,
- keep the `main` branch clean and stable.

You always create a branch from an **existing** branch — typically from the default branch of your repository (for example, `main`). Work on this new branch is isolated from changes that other collaborators make on the repository.

A branch created to develop a specific feature is commonly called a **feature branch** or **topic branch**.

The Default Branch

When you create a new repository on GitHub, it begins with a single **default branch**. This is:

- the branch shown when someone visits your repository,
- the branch that Git checks out first when the repo is cloned,
- the base branch for new pull requests and code commits (unless another branch is specified).

By default, GitHub names this branch **main** for new repositories.

You can:

- change the default branch for an existing repository,
- set the default branch name for all new repositories you create.

The GitHub Flow

1. Start from a clean `main` branch.

```
$ git switch main
```

```
A --- B --- C (*)
```

2. Create a new branch for your feature or fix.

```
$ git switch -c feature-1
```

```
A --- B --- C
       \
        D (*)
```

3. Commit your work on the feature branch.

```
$ git add ...  ; git commit -m "Work"
```

```
A --- B --- C
       \
        D --- E --- F (*)
```

Alternative 1): Merging a Branch via Pull Request

AImage Lab
University of Modena and Reggio Emilia

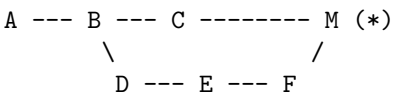
4. Push the branch to GitHub.

```
$ git push -u origin feature-1
```

```
(local -> remote sync)
feature-1 uploaded (*)
```

5a. GitHub UI (recommended workflow)

Open a pull request from `feature-1` into `main`, review the changes, then click **Merge**.

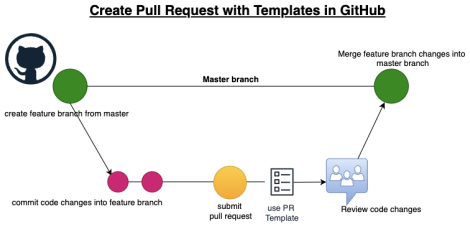


What Is a Pull Request?

A **pull request (PR)** is a GitHub feature used to propose changes from one branch into another (usually from a feature branch into `main`).

A PR allows you to:

- share your changes with collaborators,
- discuss and review the code,
- run automated tests and checks,
- request approval before merging.



A pull request is **not** a Git command. It is a workflow for collaboration and code review.

Once approved, the PR ends with a **merge** performed on GitHub.

Alternative 2): Merging a Branch Directly on `main`

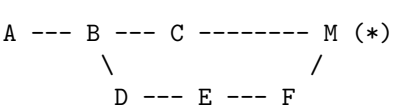
4. Push the branch to GitHub.

```
$ git push -u origin feature-1
```

```
(local -> remote sync)
feature-1 uploaded (*)
```

5b. Local merge (no pull request)

You can integrate `feature-1` directly into `main` using Git commands on your machine.



Commands

```
$ git switch main
$ git pull          # update local main
$ git merge feature-1
$ git push          # update remote main
```

Creating and Switching Branches

A new branch is created starting from the commit you are currently on. When you switch to a branch, Git prepares a **separate working directory** and a **separate staging area** for that branch.

Important: All branches in a repository share the **same local repository** (i.e., the same commit history database), but each checked-out branch has its own working copy and index.

Two ways to create a branch:

- Create the branch, then switch to it.
- Create & switch in a single step.

Create, then switch:

```
$ git branch feature-1
# branch created

$ git switch feature-1
# now on feature-1
```

Create + switch in one step:

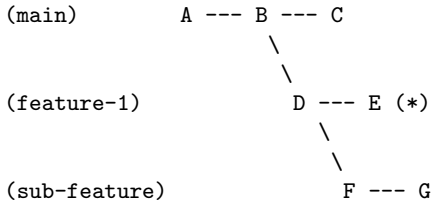
```
$ git switch -c feature-2
```

Branching from Any Branch

A branch does *not* need to be created from `main`. You can create a new branch from **any existing branch**.

This enables workflows such as:

- feature branches from `main`,
- sub-features branching from another feature branch,
- hotfix branches starting from a release branch.



Each new branch starts at the commit you were on.

Branch Naming Conventions

Git projects often adopt naming conventions to clarify the purpose and lifecycle of each branch. Common patterns include:

Main branches

- **main** – stable production-ready branch.
- **develop** – integration branch where new features are assembled (common in Git Flow).

Feature branches

- `feature/<name>` – used to implement new functionality in isolation.

Release branches

- `release/<version>` – preparation for a new release (testing, fixes, documentation).

Hotfix branches

- `hotfix/<issue>` – urgent fixes applied directly to the production branch.

Working on a Branch

Once you switch to a branch, all Git operations (`git add`, `git commit`, `git pull`, `git push`) automatically apply to **that branch**.

Git knows which branch you are currently on, so you do not need to repeat the branch name for every command.

Important: You always work on a **local branch**. The remote branch on GitHub is only updated when you push.

Commands when on `feature-1` (*)

```
$ git add .
$ git commit -m "Work"
$ git push    # pushes the local branch
$ git pull    # pulls into the local branch
```

Synchronization with GitHub requires a connection between the local branch and its corresponding remote branch.

Local and Remote Branches

Each branch in your project may exist in two places:

Local branch

- stored on your machine,
- you commit and edit here,
- you always work on the local copy of a branch.

Remote branch

- stored on GitHub under the `origin` remote,
- used to share work with collaborators,
- updated only when you run `git push`.

Important. A new branch initially exists only locally. It appears on GitHub only after the first `git push`.

Tracking Branches and the First Push/Pull

For Git to synchronize branches automatically, the local branch must know which remote branch it corresponds to. This relationship is called an **upstream** or **tracking** branch.

Before the first push, the branch exists only locally:

- `git pull` cannot pull from a non-existent remote branch,

First push: create the remote branch and set tracking

- `git push -u origin feature-1`
- This creates `origin/feature-1` on GitHub (if it does not exist) and links it to the local `feature-1` branch.

After tracking is set:

```
$ git push   # Git knows where to push
$ git pull   # Git knows where to pull from
```

Bringing Changes from `main` into Your Branch

When collaborators push new commits to `main`, your feature branch may fall behind. To stay up to date, you merge `main` into your branch.

Step 1: Switch to your branch

```
$ git switch feature-1
```

Step 2: Merge the latest changes from `main`

```
$ git merge main
```

This operation **does not delete your branch**. It simply integrates commits from `main` into it.

What Happens During a Merge

Before merging: your branch is behind

```
main:         A --- B --- C
                    \
feature-1:           D --- E   (*)
```

Git analyzes both histories and finds the common ancestor

```
common ancestor = B          main changes    = C          feature changes = D, E
```

After merging: Git creates a merge commit

```
main:         A --- B --- C
                    \       \
feature-1:           D --- E --- M   (*)
```

The merge commit **M** combines the work from both branches. Your branch remains intact — it is simply updated.

Cleaning Up Branches

After merging a feature branch, keeping it around is usually unnecessary. Removing old branches keeps the repository tidy.

Delete the local branch:

```
$ git branch -d feature-1
```

Delete the remote branch on GitHub:

```
$ git push origin --delete feature-1
```

Git never deletes branches automatically. You remove them only when they are no longer needed.

Protected Branches on GitHub

Teams often protect important branches (e.g. `main`) to enforce code quality and prevent accidental changes.

Common protection rules:

- require pull requests instead of direct pushes,
- require one or more reviewer approvals,
- require status checks (CI tests) to pass,
- prevent force-pushes and branch deletions,
- require branches to be up-to-date before merging.

Protected branches ensure that the `main` branch remains stable, reviewed, and tested.

IDEs

Git Integration in Eclipse

Eclipse provides built-in support for Git through the **EGit** plugin.

Git integration in Eclipse typically involves three steps:

- **Installation and setup.** Install or enable EGit, configure Git user information.
- **Create a project with GitHub and Eclipse.** Clone a repository from GitHub or create a new project and share it using Git.
- **Use EGit to track a change.** Modify files, stage them, commit the changes, and push to the remote repository.

EGit provides a graphical interface for common Git operations directly inside Eclipse.

Installation and Setup (EGit + GitHub)

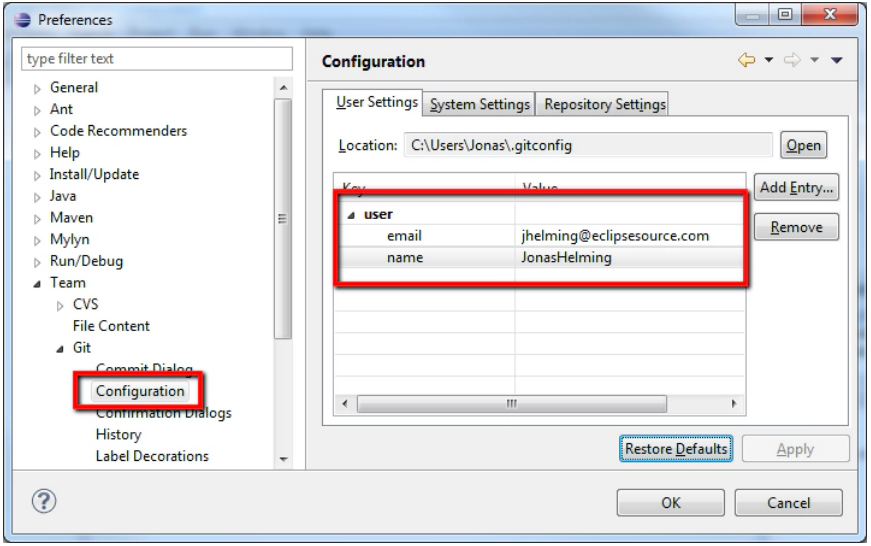
EGit is the Git integration plugin for Eclipse. It comes pre-installed in recent Eclipse versions; older versions may require manual installation.

Steps:

- **Install or verify EGit** Ensure your Eclipse distribution includes EGit, or install it if using an older version.
- **Create a GitHub account** Sign up for a free account at `github.com`.
- **Configure EGit with your GitHub identity**
  - Go to **Window** → **Preferences**.
  - Search for "git", then navigate to **Team** → **Git** → **Configuration**.
  - Click **Add Entry...**.
  - Add:
    - `user.name` = your GitHub username
    - `user.email` = your GitHub email

EGit will now use your GitHub credentials for Git operations inside Eclipse.

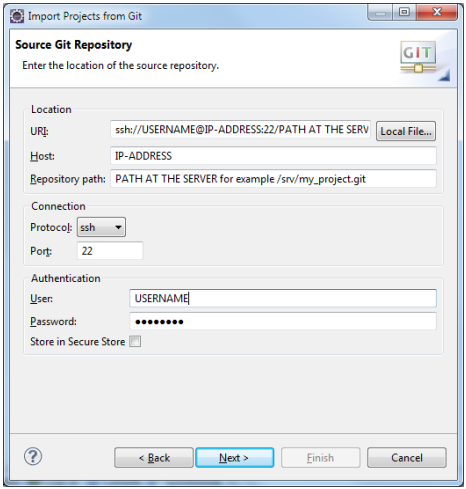
Installation and Setup



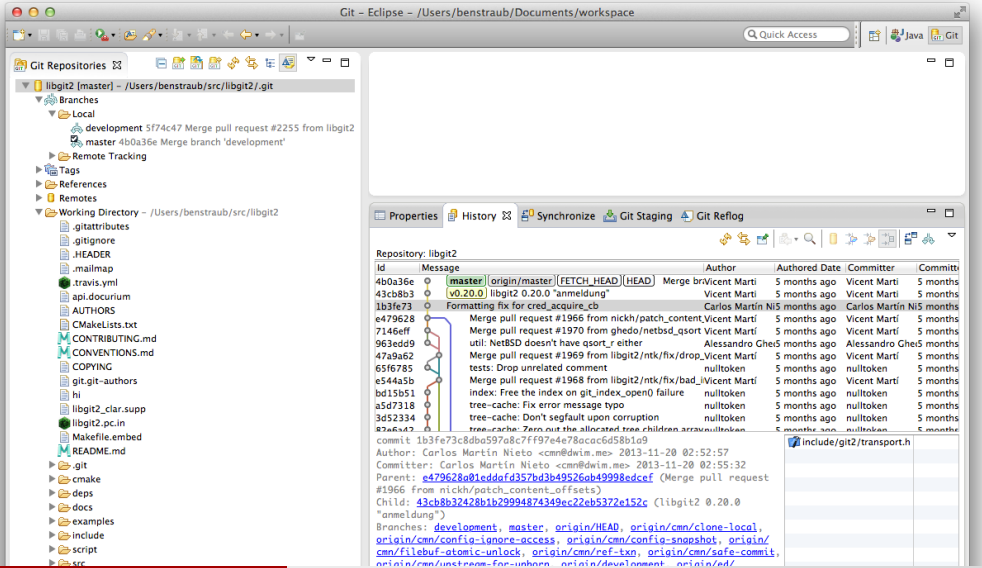
Once You Have Created Your GitHub Repo

Import the repository into Eclipse

- Open Eclipse with EGit installed (older versions may not include it).
- Go to **File → Import...**
- In the dialog, select **Git → Projects from Git** and click **Next**.
- Choose **Clone URI**, then click **Next**.
- Eclipse automatically fills in the repository details from the GitHub URL. Verify your GitHub **username** and **password** under **Authentication**, then click **Next**.



Git and IDEs


Git – Eclipse – /Users/benstraub/Documents/workspace
Quick Access
Java
Git
Git Repositories
libgit2 [master] - /Users/benstraub/src/libgit2/.git
Branches
Local
development 5f74c47 Merge pull request #2255 from libgit2
master 4b0a36e Merge branch 'development'
Remote Tracking
Tags
References
Remotes
Working Directory - /Users/benstraub/src/libgit2
.gitattributes
.gitignore
.HEADER
.mailmap
.travis.yml
api.docurium
AUTHORS
CMakeLists.txt
CONTRIBUTING.md
CONVENTIONS.md
COPYING
git.git-authors
hi
libgit2_clar.supp
libgit2.pc.in
Makefile.embed
README.md
.git
cmake
deps
docs
examples
include
script
src
Properties
History
Synchronize
Git Staging
Git Reflog
Repository: libgit2
Id
Message
Author
Authored Date
Committer
master
origin/master
FETCH_HEAD
HEAD
v0.20.0
4b0a36e
43cb8b3
1b3fe73
e479628
7146eff
963edd9
47a9a62
65f6785
e544a5b
bd15b51
a5d7318
3d52334
libgit2 0.20.0 "anmeldung"
Formatting fix for cred_acquire_cb
Merge pull request #1966 from nickh/patch_content_
Merge pull request #1970 from ghedo/netbsd_qsort
util: NetBSD doesn't have qsort_r either
Merge pull request #1969 from libgit2/ntk/fix/drop_
tests: Drop unrelated comment
Merge pull request #1968 from libgit2/ntk/fix/bad_i
index: Free the index on git_index_open() failure
tree-cache: Fix error message typo
tree-cache: Don't segfault upon corruption
Vicent Martí
Carlos Martín Ni
Alessandro Ghe
nulltoken
5 months ago
commit 1b3fe73c8dba597a8c7ff97e4e78acac6d58b1a9
Author: Carlos Martín Nieto <cmn@dwim.me> 2013-11-20 02:52:57
Committer: Carlos Martín Nieto <cmn@dwim.me> 2013-11-20 02:55:32
Parent: e479628a01eddafd357bd3b49526ab49998edcef (Merge pull request #1966 from nickh/patch_content_offsets)
Child: 43cb8b32428b1b29994874349ec22eb5372e152c (libgit2 0.20.0 "anmeldung")
Branches: development, master, origin/HEAD, origin/cmn/clone-local, origin/cmn/config-ignore-access, origin/cmn/config-snapshot, origin/cmn/filebuf-atomic-unlock, origin/cmn/ref-txn, origin/cmn/safe-commit,
include/git2/transport.h

Git Integration in Eclipse — References

Useful resources for learning and using Git and EGit within Eclipse:

- https://eclipsesource.com/blogs/tutorials/egit-tutorial/
- http://www.vogella.com/tutorials/EclipseGit/article.html
- https://wiki.eclipse.org/EGit/User_Guide
- http://www.geo.uzh.ch/microsite/reproducible_research/post/rr-eclipse-git/

These guides cover installation, configuration, workflows, and advanced features of EGit in Eclipse.

Summary

Summary (1/2)

1. Version Control Systems (VCS)

- Track changes to files over time.
- Facilitate collaboration while preserving history.
- Enable branching, rollback, conflict resolution, and safe experimentation.

2. Git and GitHub Architecture

- **Git**: a distributed VCS running locally on your machine.
- Local repository = working directory + staging area + commit history.
- **GitHub**: cloud hosting + collaboration tools (PRs, issues, etc.).
- Synchronization between local and remote via `push`, `pull`, `fetch`.

Summary (2/2)

3. Core Git Features

- **Branching**: isolate work, develop features safely.
- **Merging**: integrate branches; via PR or local merge.
- **Pull Requests**: code review, discussion, CI checks.
- **History inspection**: `git log`, `git status`, `git diff`.
- **Remote collaboration**: tracking branches, resolving conflicts.

4. IDE Integration

- Tools like **EGit** (Eclipse) simplify Git operations.
- Clone, stage, commit, push, and manage branches visually.
- IDEs help beginners adopt Git more easily while remaining compatible with command-line workflows.